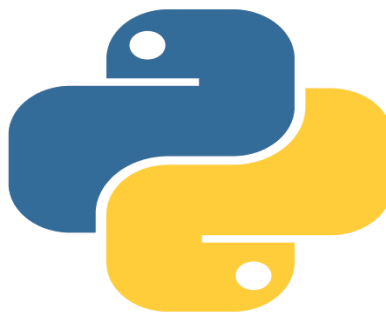




Python

Part 5

Abstraction of class

- Data abstraction means showing only the essential features and hiding the complex internal details.
- Technically, in Python abstraction is used to hide the implementation details from the user and expose only necessary parts, making the code simpler and easier to interact with.

A smartphone is a great real-life example of data abstraction you can make calls or take photos without knowing how signals or storage work. Only essential features are shown, complex details are hidden.

Abstract classes are created using abc module and @abstractmethod decorator, allowing developers to enforce method implementation in subclasses while hiding complex internal logic.

Example:

```
from abc import ABC, abstractmethod
class Greet(ABC):
    @abstractmethod
    def say_hello(self):
        print("hello world")
class English(Greet):
    def say_hello(self):
        return "Hello!"
g = English()
print(g.say_hello())
```

Output:

```
Hello!
```

Encapsulation of class

Encapsulation means hiding internal details of a class and only exposing what's necessary. It helps to protect important data from being changed directly and keeps the code secure and organized.

1. Public Members

Public members are variables or methods that can be accessed from anywhere inside the class, outside the class or from other modules. By default, all members in Python are public.

They are defined without any underscore prefix (e.g., `self.name`).

Example: This example shows how a public attribute (`name`) and a public method (`display_name`) can be accessed from outside the class using an object.

```
class Employee:
    def __init__(self, name):
        self.name = name # public attribute

    def display_name(self): # public method
        print(self.name)
```

```
emp = Employee("John")
emp.display_name() # Accessible
print(emp.name)    # Accessible
```

Output:

```
John
John
```

Explanation:

- **self.name:** Declared without underscores, so it is public.
- **display_name():** Public method that prints the value of the public attribute.
- **emp.name:** Directly accessed from outside the class, showing public members are fully accessible.

2. Protected members

Protected members are variables or methods that are intended to be accessed only within the class and its subclasses. They are not strictly private but should be treated as internal.

In Python, protected members are defined with a single underscore prefix (e.g., `self._name`).

Example: This example shows how a protected attribute (**`_age`**) can be accessed within a subclass, demonstrating that protected members are meant for use within the class and its subclasses.

```
class Employee:

    def __init__(self, name, age):

        self.name = name    # public

        self._age = age     # protected

class SubEmployee(Employee):

    def show_age(self):
```

```
print("Age:", self._age) # Accessible in subclass

emp = SubEmployee("Ross", 30)

print(emp.name)      # Public accessible

emp.show_age()       # Protected accessed through subclass
```

Output:

```
Ross
Age: 30
```

Explanation:

- **self._age:** Defined with a single underscore, marking it as protected.
- **SubEmployee:** Inherits from Employee and can access _age directly.
- Protected members should not be accessed outside the class hierarchy, but Python does not enforce this rule strictly.

3. Private members

Private members are variables or methods that cannot be accessed directly from outside the class. They are used to restrict access and protect internal data.

In Python, private members are defined with a double underscore prefix (e.g., self.__salary). Python applies [name mangling](#) by internally renaming them (e.g., __salary becomes _ClassName__salary) to prevent direct access.

Example: This example shows how a private attribute (**__salary**) is accessed within the class using a public method, demonstrating that private members cannot be accessed directly from outside the class.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name      # public
        self.__salary = salary # private

    def show_salary(self):
        print("Salary:", self.__salary)
```

```
emp = Employee("Robert", 60000)
print(emp.name)      # Public accessible
emp.show_salary()    # Accessing private correctly
# print(emp.__salary) # Error: Not accessible directly
```

Output:

```
Robert
Salary: 60000
```

Explanation:

- **self.__salary:** Defined with double underscores, so it is private.
- **show_salary():** A public method that provides safe access to the private attribute.
- Attempting **emp.__salary** causes an `AttributeError`, proving private members cannot be accessed directly.

Exception handling

Python Exception Handling allows a program to gracefully handle unexpected events (like invalid input or missing files) without crashing. Instead of terminating abruptly, Python lets you detect the problem, respond to it, and continue execution when possible.

Example:

```
x=10
try:
    print(x)
except:
    print("error")
else:
    print("else statement")
finally:
    print("finally statement")
```

Decorators in Python

In Python, decorators are flexible way to modify or extend behavior of functions or methods, without changing their actual code.

- A decorator is essentially a function that takes another function as an argument and returns a new function with enhanced functionality.
- Decorators are often used in scenarios such as logging, authentication and memorization, allowing us to add additional functionality to existing functions or methods in a clean, reusable way.

Example:

```
def decorator(func):  
    def wrapper():  
        print("Before calling the function.")  
        func()  
        print("After calling the function.")  
    return wrapper  
  
@decorator # Applying the decorator to a function  
def greet():  
    print("Hello, World!")  
greet()
```

Output:

```
Before calling the function.  
Hello, World!  
After calling the function.
```

Example:

```
def decor(func):  
    def wrap():  
        print(".....")  
        func()  
        print(".....")  
    return wrap  
  
@decor  
def text():  
    print("hiii")  
  
text()
```

Output:

```
.....  
hiii  
.....
```

Type casting in python

Type Casting is the method to convert the Python variable [datatype](#) into a certain data type in order to perform the required operation by users.

Implicit Type Conversion

In this, method, [Python](#) converts the datatype into another datatype automatically. Users don't have to involve in this process.

Example:

```
# implicit type Casting  
  
# Python automatically converts  
# a to int  
a = 7  
print(type(a))
```



```
# Python automatically converts
# b to float
b = 3.0
print(type(b))

# Python automatically converts
# c to float as it is a float addition
c = a + b
print(c)
print(type(c))

# Python automatically converts
# d to float as it is a float multiplication
d = a * b
print(d)
print(type(d))
```

Output:

```
<class 'int'>
<class 'float'>
10.0
<class 'float'>
21.0
<class 'float'>
```

Explicit Type Conversion

In this method, Python needs user involvement to convert the variable data type into the required data type.

Examples of Type Casting in Python

Mainly type casting can be done with these data type functions:

- **Int():** [Python Int\(\)](#) function take float or string as an argument and returns int type object.
- **float():** [Python float\(\)](#) function take int or string as an argument and return float type object.
- **str():** [Python str\(\)](#) function takes float or int as an argument and returns string type object.

Converting Int to Float in Python with the **float()** function.

Example:

```
# int variable  
a = 5  
  
# typecast to float  
n = float(a)  
  
print(n)  
print(type(n))
```

Output:

```
5.0  
<class 'float'>
```

Converting Float to int datatype in Python with int() function.

Example:

```
# int variable  
a = 5.9  
  
# typecast to int  
n = int(a)  
  
print(n)  
print(type(n))
```

Output:

```
5  
<class 'int'>
```

Converting int to String datatype in Python with `str()` function

Example:

```
# int variable  
a = 5  
  
# typecast to str  
n = str(a)  
  
print(n)  
print(type(n))
```

Output:

```
5  
<class 'str'>
```

Python Convert string to int

Example:

```
# string variable  
a = "5"  
b = 't'  
  
# typecast to int  
n = int(a)  
  
print(n)  
print(type(n))  
  
print(int(b))  
print(type(b))
```

Output:

```
5
<class 'int'>
-----
ValueError                                Traceback (most recent call last)
/tmp/ipython-input-1957009883.py in <cell line: 0>()
      9 print(type(n))
     10
--> 11 print(int(b))
     12 print(type(b))
ValueError: invalid literal for int() with base 10: 't'
```

Enumerate() in Python

enumerate() function adds a counter to each item in a list or any other iterable, and returns a list of tuples containing the index position and the element for each element of the iterable.

Example:

```
a = ["python", "c", "java"]

# Iterating list using enumerate to get both index and element
for i, name in enumerate(a):
    print(f"Index {i}: {name}")

# Converting to a list of tuples
print(list(enumerate(a)))
```

Output:

```
Index 0: python
Index 1: c
Index 2: java
[(0, 'python'), (1, 'c'), (2, 'java')]
```

